



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura:

Estructura de Datos y Algoritmos II

Proyecto final:

Software para la visualización y estudio de algoritmos y estructuras de datos
-Backend-

Documentación Técnica del Módulo Core

Análisis, Diseño e Implementación de Endpoints REST en el
Backend

Carpeta documentada:

`app/core/`

Equipo de Desarrollo de Backend

18 de mayo de 2026

1. Módulo Core

La carpeta `core` contiene componentes fundamentales para el funcionamiento interno del backend. Su propósito principal es definir la conexión con la base de datos, declarar modelos persistentes y proporcionar funciones básicas para la lectura de archivos.

Este módulo no está orientado directamente a la parte visual del sistema, pero sirve como soporte para almacenar información, recuperar datos y mantener registros que posteriormente pueden ser consultados o mostrados desde el front-end.

1.1. Estructura General

La carpeta `core` contiene los siguientes archivos principales:

```
core/  
  database.py  
  ingesta.py  
  models.py
```

El archivo `__init__.py` no se considera en esta documentación debido a que se encuentra vacío.

1.2. Archivo `database.py`

El archivo `database.py` configura la conexión entre el backend y la base de datos. En este caso, se utiliza SQLite mediante SQLAlchemy.

Elemento	Descripción breve	Uso en front-end
<code>DATABASE_URL</code>	Define la ubicación de la base de datos SQLite usada por el sistema.	No se muestra directamente, pero permite que la información del sistema pueda persistir.
<code>engine</code>	Crea el motor de conexión hacia la base de datos.	Permite que el backend consulte o guarde información que después puede visualizarse.
<code>SessionLocal</code>	Genera sesiones para interactuar con la base de datos.	Soporta operaciones de consulta, registro o actualización desde acciones realizadas en la interfaz.
<code>Base</code>	Base declarativa para definir modelos de tablas con SQLAlchemy.	Permite estructurar los datos que eventualmente serán presentados al usuario.
<code>get_db()</code>	Abre y cierra sesiones de base de datos de forma controlada.	Puede usarse en endpoints que necesiten obtener información para mostrarla en pantalla.

Cuadro 1: Resumen del archivo `database.py`

En términos generales, este archivo funciona como la configuración central de persistencia. Gracias a él, otros módulos pueden conectarse a la base de datos sin repetir la configuración en cada archivo.

1.3. Archivo `models.py`

El archivo `models.py` define los modelos de datos que serán representados como tablas en la base de datos. Estos modelos permiten guardar información relacionada con conversaciones y ejecuciones de algoritmos.

Modelo	Descripción breve	Uso en front-end
<code>ChatHistory</code>	Representa mensajes de una conversación, incluyendo sesión, contenido, emisor y fecha.	Puede usarse para mostrar historial de chat o actividad previa del usuario.
<code>AlgorithmExecution</code>	Registra ejecuciones de algoritmos, incluyendo nombre, fecha, estado, parámetros y resultado.	Puede mostrarse como historial de ejecuciones, bitácora o panel de resultados anteriores.

Cuadro 2: Resumen del archivo `models.py`

El modelo `ChatHistory` permite almacenar mensajes asociados a una sesión. Esto puede ser útil si el sistema incluye una sección conversacional, asistente o historial de interacción.

El modelo `AlgorithmExecution` permite registrar qué algoritmo se ejecutó, cuándo se ejecutó, con qué parámetros y cuál fue el resultado. Esto puede servir para crear una vista de historial donde el usuario consulte operaciones anteriores.

1.4. Archivo `ingesta.py`

El archivo `ingesta.py` contiene funciones simples para leer información desde archivos externos en diferentes formatos.

Función	Descripción breve	Uso en front-end
<code>leer_txt(path)</code>	Lee un archivo de texto y devuelve sus líneas sin saltos de línea.	Puede permitir que el usuario cargue archivos de texto para analizarlos o mostrarlos.
<code>leer_csv(path)</code>	Lee un archivo CSV y devuelve sus filas como listas.	Puede usarse para mostrar datos en tablas dentro de la interfaz.
<code>leer_json(path)</code>	Lee un archivo JSON y devuelve su contenido estructurado.	Puede usarse para cargar configuraciones, datos o resultados en formato estructurado.

Cuadro 3: Resumen del archivo `ingesta.py`

Este archivo puede servir como soporte para futuras funciones de importación de datos. Desde el front-end, el usuario podría subir un archivo y el backend podría leerlo para mostrar su contenido o procesarlo con algún algoritmo.

2. Resumen General de Uso en Front-End

El módulo `core` no representa una vista visual por sí mismo, sino que funciona como soporte interno para otras partes del sistema. Sin embargo, sus datos pueden utilizarse para construir secciones visibles dentro del front-end.

Archivo	Función principal	Vista sugerida en front-end
<code>database.py</code>	Configura la conexión con la base de datos.	No requiere vista propia; da soporte a vistas que consultan o guardan información.
<code>models.py</code>	Define tablas para historial de chat y ejecuciones de algoritmos.	Historial de actividad, historial de ejecuciones o panel de registros.
<code>ingesta.py</code>	Lee archivos TXT, CSV y JSON.	Vista para cargar archivos y mostrar su contenido procesado.

Cuadro 4: Resumen visual del módulo core

El flujo general sería:

1. El usuario realiza una acción desde el front-end.
2. El backend procesa la solicitud mediante un endpoint.
3. Si es necesario, el backend usa **core** para consultar, guardar o leer información.
4. La información procesada se devuelve al front-end.
5. El front-end muestra los datos como historial, tabla, tarjeta o resultado.

3. Conclusión

La carpeta **core** funciona como una base de soporte para el backend. Aunque no contiene directamente algoritmos visuales ni endpoints principales, sí proporciona elementos esenciales para el funcionamiento del sistema, como la conexión a base de datos, los modelos de almacenamiento y la lectura de archivos.

Esta documentación sintetizada permite identificar rápidamente qué contiene cada archivo y cómo puede apoyar a la parte visual del sistema, especialmente en funciones como historial de ejecuciones, almacenamiento de mensajes, consulta de resultados previos y carga de datos desde archivos.